Managed by ASP.NET Core Identity	Requires implementing by the developer
Database schema for storing users and claims	UI for logging in, creating, and managing users (Razor Pages or controllers); included in an optional package that provides a default UI
Creating a user in the database	Sending email messages
Password validation and rules	Customizing claims for users (adding new claims)
Handling user account lockout (to prevent brute-force attacks)	Configuring third-party identity providers
Managing and generating MFA/2FA codes	Integration into MFA such as sending SMS messages, time-based one-time password (TOTP) authenticator apps, or hardware keys
Generating password-reset tokens	
Saving additional claims to the database	
Managing third-party identity providers (for example, Facebook, Google, and Twitter)	

The biggest missing piece is the fact that you need to provide all the UI for the application, as well as tying all the individual Identity services together to create a functioning sign-in process. That's a big missing piece, but it makes the Identity system extremely flexible.

Luckily, ASP.NET Core includes a helper NuGet library, Microsoft.AspNetCore.Identity.UI, that gives you the whole of the UI boilerplate for free. That's over 30 Razor Pages with functionality for logging in, registering users, using 2FA, and using external login providers, among other features. You can still customize these pages if you need to, but having a whole login process working out of the box, with no code required on your part, is a huge win. We'll look at this library and how you use it in sections 23.3 and 23.4.

For that reason, I strongly recommend using the default UI as a starting point, whether you're creating an app or adding user management to an existing app. But the question remains as to when you should use Identity and when you should consider rolling your own.